This is one of the most important backend concepts to understand. Most beginners learn CRUD APIs, but in interviews they struggle when asked:

- **Why did you use `find()` instead of `aggregate()`?**
- **Why is this API designed this way?**
- **When should you use aggregation?**
- **How do real companies build APIs?**

Let's learn it as if you're becoming a backend developer at a product company.

How a Backend API Works

Imagine you're building an e-commerce application.

```
React Frontend
      |
      | HTTP Request
      ▼
Express Route
      |
      ▼
Controller
      |
      ▼
Business Logic
      |
      ▼
MongoDB Query
      |
      ▼
Database
      |
      ▼
JSON Response
      |
      ▼
React UI
```

Example

```
User clicks "Mobiles"

↓

GET /api/products?category=Mobile

↓
```

```
Controller receives request

↓

Build query

↓

MongoDB finds products

↓

Return JSON

↓

React displays products
```

This is the complete request flow.

Types of APIs

In real projects, you won't use only CRUD. APIs generally fall into these categories:

1. CRUD APIs (Most Common)

Used for creating and managing data.

```
POST /products
```

Create product.

```
GET /products
```

Read products.

```
PUT /products/:id
```

Update product.

```
DELETE /products/:id
```

Delete product.

Used in: Admin panels, ERP systems, CMS, HRMS, CRM.

---

2. Listing APIs ⭐ (Most Used)

Most frontend screens display lists.

Example:

```
Amazon Home
```

```
Nike Shoes
```

```
Employee List
```

```
Orders
```

These use:

```
find()
```

Example:

```javascript
Product.find(filter)
.sort(sort)
.skip(skip)
.limit(limit)
```

Why?

Because you only want documents.

No calculations.

---

3. Detail API

When the user opens a single item.

```
GET /products/65abc
```

Use

```javascript
```

```
findById()
```

or

```
JavaScript
findOne()
```

---

4. Search API

Example

```
Amazon Search

iphone
```

Use

```
JavaScript
Product.find({

title:{
$regex:"iphone",
$options:"i"
}

})
```

Still uses

```
find()
```

No aggregation needed.

---

5. Filter API

Example

```
Price

1000-5000

Category
```

```
Laptop

Brand

Dell
```

Use

```JavaScript
find()
```

because you're filtering documents.

---

6. Pagination API

```
Page 1

Page 2

Page 3
```

Use

```JavaScript
skip()

limit()
```

Example

```JavaScript
Product.find()
.skip(20)
.limit(10)
```

---

7. Sorting API

```
Lowest Price
```

```
Highest Rating

Newest
```

Use

```JavaScript
sort()
```

Example

```JavaScript
Product.find()

.sort({

price:1

})
```

So When Do We Use Aggregate?

This is where many interviews focus.

Aggregation is used when MongoDB needs to **calculate, group, join, transform, or summarize** data—not just return documents.

Think of it like SQL's `GROUP BY`, `COUNT`, `SUM`, `AVG`, and `JOIN`.

Example 1

Amazon Dashboard

```
Total Products

Average Price

Total Stock

Highest Price
```

Normal query?

❌ Impossible.

Need

```
aggregate()
```

Example

JavaScript

```
Product.aggregate([
{
$group:{
_id:null,
totalProducts:{$sum:1},
averagePrice:{$avg:"$price"},
totalStock:{$sum:"$stock"}
}
}
])
```

Example 2

Products per Category

```
Mobile

120

Laptop

45

Shoes

90
```

Need

```
aggregate()
```

Because

```
GROUP BY category
```

Example 3

Monthly Sales

```
January
₹20L
February
₹18L
March
₹35L
```

Need

```
aggregate()
```

because

```
Group by Month
```

Example 4

Top Selling Products

```
iPhone
500 sold
Samsung
430 sold
```

Need

```
aggregate()
```

Example 5

Revenue by Brand

```
Apple

₹8 Crore

Samsung

₹6 Crore
```

Need

```
aggregate()
```

Example 6

Average Rating

```
Electronics

4.6

Shoes

4.3
```

Need

```
aggregate()
```

Simple Rule to Remember

Use `find()` when:

✔ Get documents

✔ Search

✔ Filter

✔ Pagination

✔ Sorting

✔ Product listing

✔ Employee listing

✔ User listing

✔ Orders

Example:

JavaScript

```
Product.find({
category:"Laptop"
})
```

Use `aggregate()` when:

✔ Count

✔ Sum

✔ Average

✔ Maximum

✔ Minimum

✔ Group

✔ Dashboard

✔ Analytics

✔ Reports

✔ Charts

✔ Revenue

✔ Monthly Reports

What About `populate()`?

Imagine

```
Orders Collection

{

user:ObjectId

product:ObjectId

}
```

You want

```
User Name

Product Name
```

Use

```JavaScript
populate()
```

Example

```JavaScript
Order.find()

.populate("user")

.populate("product")
```

This is suitable for simple relationships.

When Do We Use `$lookup`?

Suppose you need to:

- Join collections
- Filter joined data
- Calculate totals after joining
- Build analytics or reports

Then use `aggregate()` with `$lookup`.

Example:

JavaScript

```
Order.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "user",
      foreignField: "_id",
      as: "user"
    }
  }
]);
```

Real Project Example (Amazon)

Home Page

```
Latest Products
```

Use

```
find()
```

Search

```
iphone
```

Use

```
find()
```

Filters

```
Price

Brand

Category
```

Use

```
find()
```

Product Detail

Use

```
findById()
```

Admin Dashboard

```
Total Orders

Today's Revenue

Total Customers

Average Rating
```

Use

```
aggregate()
```

Sales Report

```
Revenue

Last 12 Months
```

Use

```
aggregate()
```

Interview Question

Interviewer: Why did you use `find()` instead of `aggregate()`?

A good answer would be:

> "The API only needs to return product documents with filtering, searching, sorting, and pagination. These operations are efficiently handled by `find()`. Aggregation is more appropriate when we need grouped results, calculations, joins with complex transformations, or analytics such as dashboards and reports."

Industry Decision Table

Requirement	Method
Get all products	`find()`
Search products	`find()` + `$regex`
Filter by category	`find()`
Sort by price	`find().sort()`
Pagination	`find().skip().limit()`
Get one product	`findById()`
Count products	`countDocuments()`
Dashboard statistics	`aggregate()`
Monthly sales report	`aggregate()`
Revenue report	`aggregate()`
Category-wise count	`aggregate()`
Brand-wise analytics	`aggregate()`
Get order with user details	`populate()` (simple) or `aggregate()` + `$lookup` (advanced)

A learning roadmap for interviews

Since you're preparing for backend interviews, I recommend learning MongoDB queries in this order:

1. Basic CRUD (`find`, `findOne`, `findById`, `create`, `update`, `delete`)

2. Filtering (`$gte`, `$lte`, `$in`, `$or`, `$and`, `$regex`)
3. Sorting and pagination (`sort`, `skip`, `limit`)
4. Indexing and performance basics
5. `populate()` for relationships
6. Aggregation pipeline (`$match`, `$group`, `$project`, `$lookup`, `$unwind`, `$sort`, `$facet`)
7. Building real-world APIs (products, orders, users, analytics)
8. Performance optimization and query design

If you master these topics, you'll be able to confidently explain not just *how* your APIs work, but *why* you chose a particular approach, which is exactly what interviewers look for.